Pimpri-Chinchwad Education Trust's
**Pimpri-Chinchwad College of Engineering**
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

# Assignment No 2

**Name:** Soham Kulkarni
**PRN:** 124B1F056
**Subject:** Design and Analysis of Algorithms
**Batch:** A3
**GitHub Link:** https://github.com/SOHAM0007-CODER/DAA_ASSIGNMNET

## 1. Topic

Optimize an OTT movie recommendation system by sorting movie records with Quick Sort. Let the user choose rating, release year or popularity. Read records from a CSV file and display the highest-ranked movies for the selected preference.

## 2. Course Objective

- Understand the computational complexity of algorithms.
- Select a suitable algorithm design strategy for a practical problem.

## 3. Course Outcome

- Apply Divide and Conquer to sorting practical records.
- Analyze pivot selection, recursion cost and the performance of Quick Sort.

## Theory

Quick Sort selects a pivot, partitions the array around it, and recursively sorts the parts on either side. The supplied program chooses the last movie as pivot and uses a comparator for rating, year or popularity. Each partition puts the pivot into its final sorted position.

The comparators sort values in ascending order internally. The program then prints from the end to show the top ten in descending order. Equal-key records need not retain their original order because this Quick Sort is not stable. No separate merge step is needed.

## Working of the Algorithm

- 1. Load movie title, rating, release year and popularity from movies.csv.
- 2. Read the requested sort key and choose its comparator.
- 3. Partition using the last element as pivot; recursively sort the left and right parts.
- 4. Print the highest ten values and measure the sorting time.

## Complexity Analysis

A partition scans its current subarray once. Balanced partitions give about log N recursion levels with O(N) work per level. Highly unbalanced partitions can produce N levels and quadratic work.

## Time Complexity

**Best Case:** O(N log N), when partitions remain balanced.

**Average Case:** O(N log N) for typical randomly ordered data.

**Worst Case:** O(N squared), possible with repeated or unfavorable pivot choices.

## Space Complexity

Partitioning is in place. Recursion uses O(log N) stack space on average and O(N) in the worst case. The vector holding movie records uses O(N) total data storage.

## Algorithm - Steps

- Step 1: Read the dataset and selected preference.
- Step 2: Choose a pivot, partition, and recursively sort both sides.
- Step 3: Traverse the sorted result backwards to display the highest values.

## Pseudocode

```
QUICK_SORT(A, low, high):
    if low >= high: return
    pivot = A[high]; p = low
    for j from low to high-1:
        if A[j] comes before pivot:
            swap A[j], A[p]; p = p+1
    swap A[p], A[high]
    QUICK_SORT(A, low, p-1)
    QUICK_SORT(A, p+1, high)
```

## Flow of Execution

START -> Read CSV -> Choose sort key -> Partition and recurse -> Display top records -> END.

## 6. Example

For illustrative ratings [3, 1, 4, 2], choose pivot 2. Partition to [1, 2, 4, 3]. Sorting the right part gives [1, 2, 3, 4]. Reading backwards displays 4, 3, 2, 1, with the highest rating first.

Randomized pivots or median-of-three selection can reduce sensitivity to input order, but do not remove the general quadratic worst case. These are possible improvements; the current source uses the last element.

## Conclusion

Quick Sort sorts movie records according to user preferences using Divide and Conquer. Average performance is O(N log N), while pivot quality affects worst-case time and recursion depth. The selected comparator makes the same sorting routine useful for several movie attributes.